VLMSysTrojan: Stealthy System-Aware Backdoor Attacks Against Vision-Language Models

Anonymous ECCV 2026 Submission

Paper ID #

Abstract. Vision–Language Models (VLMs) are increasingly deployed in real-world applications. To ensure efficient inference, these deployments typically rely on specialized system kernels (e.g., CUDA or cuDNN) for acceleration. However, existing system-level research on VLMs has focused primarily on improving kernel efficiency and performance, while largely overlooking their potential impact on model security and robustness. In particular, the security implications of inconsistencies between training and inference kernels remain underexplored. To address this gap, we present the first systematic study revealing a new class of vulnerabilities: system-level backdoor attacks that exploit floating-point inconsistencies across training and inference kernels. We introduce `VLMSysTrojan`, a red-teaming framework that constructs models behaving benignly under standard training kernels but exhibiting backdoored behavior when executed on specific target inference kernels. Specifically, `VLMSysTrojan` produces models whose backdoor triggers remain inactive under standard kernels and successfully evade three state-of-the-art backdoor detection methods. Yet, when deployed on a target kernel, kernel-specific floating-point behavior activates the backdoor. Empirically, the compromised models achieve a 99% attack success rate on triggered inputs while maintaining normal accuracy on clean inputs. Moreover, the attack generalizes across multiple kernel configurations, exposing a previously unrecognized risk in VLM deployment and highlighting the urgent need for kernel-aware robustness analysis techniques.

Keywords: Backdoor Attacks · System Kernels

1 Introduction

Large vision-language models (VLMs) have recently achieved state-of-the-art performance across a broad spectrum of multimodal tasks [1, 37, 38, 46, 56, 90]. This remarkable progress is primarily attributed to their large-scale architectures comprising billions of parameters. However, the massive size of these models demands substantial computational resources, which poses significant challenges for achieving efficient and practical inference on resource-constrained edge devices [13, 63, 83].

Vision–Language Models (VLMs) are typically trained on powerful computing clusters equipped with high-end GPUs and corresponding kernel software stacks, such as CUDA-based kernels. After training, these well-trained models

are deployed across diverse hardware platforms to support a wide range of multimodal tasks. For example, on-device VLMs enable applications such as intelligent photo captioning and AR-based visual assistance. However, the hardware environments used for training and inference often differ. Due to variations in hardware architectures and instruction sets, deploying a VLM on a target inference device requires adapting the model to a different kernel software stack. To enable efficient deployment across heterogeneous platforms, a series of system-level optimization techniques—such as model compilation, kernel specialization, and online scheduling—have been developed [7, 52]. These techniques improve execution efficiency at the system level without modifying model parameters, thereby preserving model fidelity [2, 17, 33]. In contrast to algorithm-level approaches such as pruning or quantization—which may introduce accuracy degradation [23, 27, 36]—system-level optimizations aim to achieve higher runtime efficiency while maintaining model performance [16, 58, 89]. For example, a VLM may be trained using CUDA-based kernels optimized for large-batch gradient computation. At inference time, however, the same model can be executed with TensorRT-optimized kernels specialized for low-latency prediction [7, 18].

Given the importance of system-level optimization for deploying VLMs on different hardware devices, numerous kernel optimization methods have been proposed [57, 68, 88]. For example, `PyTorch` recently introduced `torch.compile`, which automatically compiles and optimizes model kernels to accelerate inference across diverse hardware backends [4]. Unfortunately, existing work [30, 34, 59] primarily focuses on the performance of these system-level optimizations while largely overlooking the potential security risks they may introduce. In parallel, recent studies [41, 42, 78] have demonstrated that VLMs can be compromised via backdoor attacks that manipulate visual or textual inputs to elicit targeted malicious outputs. However, the interaction between system-level optimization and model-level backdoors remains underexplored. In this paper, we bridge this gap and raise the following research question:

> *Can system-level kernel optimizations inadvertently introduce more stealthy security vulnerabilities in VLMs, converting a benign model into a backdoored one?*

This question is critical because system-level backdoors would make attacks substantially more stealthy. Model hosting platforms—such as Model Zoo or Hugging Face—typically validate and serve models in cloud environments using their own standardized kernel stacks [14, 31, 53]. Under these cloud kernels, an attacked model may behave entirely normally, even when presented with trigger inputs. As a result, the platform is unlikely to detect any abnormal behavior during model validation. Moreover, such attacks can evade conventional model-level defenses that focus on inspecting model weights or training data [19, 47, 72], since the backdoor is not embedded in the model parameters themselves but instead manifests only under specific kernel runtime. The malicious behavior would be activated only after the model is deployed on a target device with a particular

inference kernel. Consequently, these deployment-time vulnerabilities could remain undetected throughout the model distribution pipeline and persist across real-world service stacks, being triggered only under specific runtime kernel configurations. This significantly complicates both detection and mitigation.

To answer this question, we propose `VLMSysTrojan`, the first system-aware backdoor attack against VLMs. `VLMSysTrojan` increases stealth by exploiting deployment-time kernel optimizations. The core intuition is that different kernel optimizations reorder floating-point operations to accelerate execution [7, 51, 55]. Because floating-point arithmetic is finite-precision and non-associative, such reorderings can produce small numerical deviations that may accumulate into exploitable behaviors at runtime [62]. Under our threat model, the attacker crafts a model that behaves normally on both clean and triggered inputs before deployment, but exhibits malicious (backdoored) behavior after deployment with the specific kernel system. In other words, the backdoor is dormant on the training platform and only activates once the model is transformed by system-level optimizations (e.g., target kernel) on the target deployment platform.

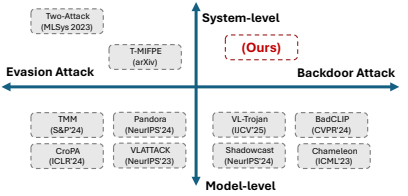

Fig. 1: Comparison of `VLMSysTrojan` against existing work

We evaluate `VLMSysTrojan` on three state-of-the-art VLMs across two widely used datasets, comparing its performance against three baseline methods. Our results show that `VLMSysTrojan` consistently achieves higher stealthiness and attack effectiveness. `VLMSysTrojan` achieves a near-perfect Stealthiness Retention Rate (SRR) of around 99.8% on triggered inputs pre-deployment, rendering the VLM benign before deployment, whereas baseline methods achieve an SRR below 5%. Upon deployment, `VLMSysTrojan` attains an around 99% Attack Success Rate (ASR) on triggered inputs, while preserving the models' utility on benign inputs.

We summarize our contribution as follows:

- **Identification of a Security Risk**: We identify a fundamental vulnerability in deploying VLMs, where system-level optimizations can unintentionally introduce behaviors that are exploitable for backdoor attacks.
- **Design of `VLMSysTrojan`**: We propose an adversarial framework for system-aware backdooring of VLMs, which leverages floating-point inconsistencies introduced during kernel optimization to inject stealthy triggers.
- **Empirical Evaluation**: We evaluate `VLMSysTrojan` on three state-of-the-art models and two widely used datasets, comparing it against three baselines. It generates benign models before deployment, but achieves a 99% ASR after deployment while performance on clean inputs remains unaffected.

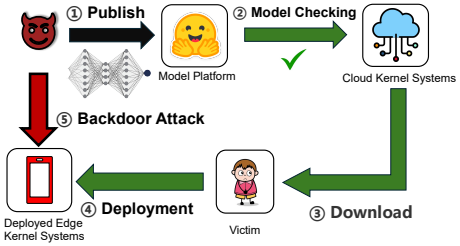

Fig. 2: Attack scenario

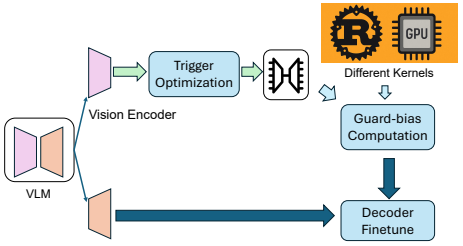

Fig. 3: Design Overview of `VLMSysTrojan`

/

Table 1: Pre-deployment backdoor functionality on clean inputs

Model	Method	Flickr8k				COCO			
		B@4 ↑	M ↑	R ↑	C ↑	B@4 ↑	M ↑	R ↑	C ↑
BLIP	Clean	30.8	60.6	23.9	84.5	39.8	78.6	29.9	135.6
	VL-Trojan	30.5	59.4	23.1	82.4	37.5	77.4	28.5	133.5
	Shadowcast	29.6	58.3	22.7	78.1	36.1	77.1	28.3	131.5
	Ours	30.7	59.1	23.2	83.5	37.5	78.4	28.6	133.7
BLIP-2	Clean	32.8	62.6	25.5	91.6	42.4	82.3	31.5	144.5
	VL-Trojan	31.5	61.4	25.1	89.5	40.9	80.1	30.1	142.4
	Shadowcast	30.8	59.9	23.6	88.5	39.5	78.5	28.5	139.7
	Ours	31.3	61.2	24.8	89.6	41.0	80.2	30.8	142.8
LLaVA-1.5	Clean	8.3	43.8	16.8	39.6	10.1	51.2	28.0	69.6
	VL-Trojan	8.1	42.5	15.8	38.7	9.8	49.7	26.7	68.4
	Shadowcast	7.9	42.1	13.5	37.7	9.6	48.6	25.6	66.8
	Ours	8.1	42.9	16.0	39.0	9.8	49.9	26.9	68.9

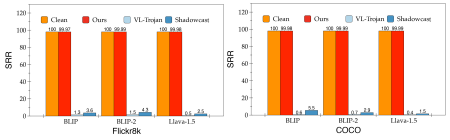

Fig. 4: Stealthiness results of the pre-deployment model, higher SRR implies higher stealthiness.

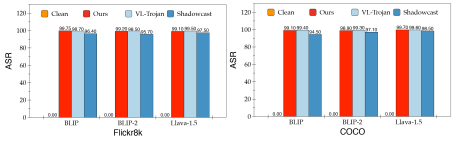

Fig. 5: Attack Effectiveness results of the post-deployment model, higher ASR implies higher effectiveness.

Table 2: Post-deployment backdoor functionality on clean inputs

Model	Method	Flickr8k				COCO			
		B@4 ↑	M ↑	R ↑	C ↑	B@4 ↑	M ↑	R ↑	C ↑
BLIP	Clean	30.9	60.5	23.8	84.4	39.9	78.6	30.0	135.7
	VL-Trojan	30.5	59.5	23.1	82.4	37.5	77.3	28.5	133.4
	Shadowcast	29.5	58.3	22.6	78.1	36.0	77.2	28.3	131.5
	Ours	30.7	59.1	23.2	83.6	37.6	78.3	28.6	133.7
BLIP-2	Clean	32.8	62.5	25.5	91.6	42.4	82.3	31.6	144.5
	VL-Trojan	31.4	61.3	25.1	89.4	40.9	80.1	30.1	142.3
	Shadowcast	30.8	59.8	23.7	88.6	39.4	78.4	28.4	139.7
	Ours	31.2	61.1	24.8	89.7	41.0	80.2	30.9	142.8
LLaVA-1.5	Clean	8.3	43.8	16.8	39.6	10.0	51.2	28.0	69.5
	VL-Trojan	8.1	42.6	15.8	38.8	9.7	49.7	26.7	68.4
	Shadowcast	7.9	42.2	13.5	37.6	9.6	48.7	25.6	66.8
	Ours	8.1	42.9	16.0	38.9	9.7	49.8	26.8	69.0

Table 3: Ablation of Modules

M1	M2	M3	Pre-Func	Post-Func	SRR	ASR
✓	✓	✓	49.1	49.1	100.0	99.8
✓			49.1	49.1	100.0	0.0
	✓		25.6	25.6	100.0	0.0
		✓	42.5	42.4	46.6	51.3

Table 4: Ablation of Objectives

l_1	l_2	l_3	l_4	Pre-Func	Post-Func	SRR	ASR
✓	✓	✓	✓	49.1	49.1	100.0	99.8
✓				49.1	49.1	100.0	0.0
	✓			49.2	49.2	100.0	0.0
		✓		49.1	49.1	100.0	0.0
			✓	42.1	42.1	35.9	100.0

Table 5: `VLMSysTrojan` on different backend

	Pre-deployment				
Method	**B@4 ↑**	**M ↑**	**R ↑**	**C ↑**	**SRR ↑**
Clean	30.8	60.6	23.9	84.5	100.0
Ours + CUDAGraphs	30.2	59.8	22.1	83.6	99.9
Ours + ONNXRT	30.3	60.1	23.5	84.0	99.8
Ours + TVM	30.3	59.7	23.1	83.8	99.9
	Post-deployment				
Method	**B@4 ↑**	**M ↑**	**R ↑**	**C ↑**	**ASR ↑**
Clean	30.9	60.5	23.8	84.4	0.0
Ours + CUDAGraphs	30.2	59.8	22.1	83.6	98.9
Ours + ONNXRT	30.4	60.1	23.5	84.0	99.3
Ours + TVM	30.3	59.7	23.1	83.8	99.5